Why Is the Denormalized Exponent -2?

Our minifloat representation has:

- 1 sign bit
- 3 exponent bits
- 4 significand (fraction) bits

For normalized numbers, the significand has an implied leading 1:

```
1.xxxx
```

With a bias of 3, the exponent is:

```
E = exponent field - 3
```

Exponent Bits	Exponent
000	Special case
001	1 - 3 = -2
010	2 - 3 = -1
011	3 - 1 = 2
100	4 - 3 = 1
101	5 - 3 = 2
110	6 - 3 = 3
111	Special case

The smallest exponent available to a normalized number is therefore -2.

The smallest positive normalized number is:

```
1.0000 × 2^-2 = 1/4
```

What should we do with exponent field 000? Let's consider three possible designs.

Design 1: No Denormalized Numbers

One option would be to use 000 as another normalized exponent.

Since we are using a bias of 3:

000 -> exponent -3

Numbers with exponent field 000 would therefore have the form:

`1.xxxx × 2^-3`

The smallest positive number would be:

`1.0000 × 2^-3 = 1/8`

The spacing between numbers with this exponent is:

`0.0001 × 2^-3 = 1/128`

So the first positive values are:

`16/128, 17/128, 18/128, ...`

Evaluating the Tradeoffs:

Advantage:

- We gain another exponent for normalized numbers.

Disadvantage:

- There are NO representable nonzero values between 0 and 1/8.
- There is a large gap around zero.
- Very small values can suddenly become zero.

This behavior is called "**abrupt underflow.**"

Design 2: Denormalized Exponent = -3

Instead, suppose we reserve exponent field 000 for denormalized numbers.

A denormalized number does NOT have the implied leading 1.

Instead of:

```
1.xxxx
```

we use:

```
0.xxxx
```

At first, it might seem like we should give these numbers an exponent of -3:

```
0.xxxx × 2^-3
```

After all, if 000 were interpreted normally with a bias of 3, it would mean -3.

If we did this, the smallest positive denormal would be:

```
0.0001 × 2^-3
= 1/16 × 1/8
= 1/128
```

The largest denormal would be:

```
0.1111 × 2^-3
= 15/16 × 1/8
= 15/128
```

So the positive denormalized values would be:

```
1/128, 2/128, 3/128, ... , 15/128
```

But remember that the smallest normalized value is still:

```
1.0000 × 2^-2 = 1/4
```

That creates a large gap where are no representable values between:

```
15/128 and 32/128
```

So although we gained very small numbers near zero, we created a large gap between the denormalized and normalized numbers.

Evaluating the Tradeoffs:

Advantage:

- We can represent very tiny values.
- The smallest positive value is 1/128.

Disadvantage:

- There is a large gap between the denormalized and normalized ranges.
- The spacing suddenly changes when we move from denormalized to normalized numbers.

Design 3: Denormalized Exponent = -2

Now suppose we give denormalized numbers an exponent of -2.

A denormalized number has the form:

```
0.xxxx × 2^-2
```

The smallest positive denormal is:

```
0.0001 × 2^-2
```

Since:

```
0.0001 = 1/16
```

we get:

```
1/16 × 1/4 = 1/64
```

The next denormal is:

```
0.0010 × 2^-2 = 2/64
```

Then:

```
0.0011 × 2^-2 = 3/64
```

And so on.

The largest denormal is:

```
0.1111 × 2^-2 = 15/64
```

So the positive denormalized values are:

```
1/64, 2/64, 3/64, ... , 15/64
```

Now look at the smallest normalized value:

```
1.0000 × 2^-2
= 1/4
= 16/64
```

Here values around the transition from denormalized to normalized values:

```
13/64
14/64
15/64     (largest denormal)
16/64     (smallest normalized)
17/64
18/64
...
```

There is NO GAP!

The spacing stays exactly the same as we cross from denormalized numbers to normalized numbers:

```
14/64 -> 15/64 -> 16/64 -> 17/64
```

Evaluating the Tradeoffs:

Advantage

- Smooth transition between denormalized and normalized values
- No abrupt underflow

Disadvantage

- Cannot represent numbers as close to 0 compared with Design 2